

redundant list/query tools.

- `AGENTIC_MAX_TOOL_ROUNDS` (default 2): hard cap on [LLM API](#) calls per user message.
- `AGENTIC_FIRE_AND_FORGET` (default on): the host dispatches an ordered tool batch without waiting for per-cell snapshot reconciliation before the model's implement round completes.

The batch executor (`agentic_batch_executor.py`) integrates with `strict_execution_engine.py` to build a deterministic queue. Bulk `delete_by_index` and `insert_cell` operations are sorted in *descending* index order on the host before browser dispatch, mitigating index-shift errors when multiple positional edits occur in one turn. `run_cell` actions are ordered after structural mutations. Tool dispatches are logged through `tool_call_terminal.py` for development-time auditing.

Scrape-based observers include `cell_execution_observer.py` and `cell_structure_observer.py`. They update operational state asynchronously when cells run or the notebook layout changes, supplying feedback for monitors without a closed-loop replanning cycle in the current implementation.

3.6 LLM Integration: GLM 4.7 via Cerebras

Remote inference uses the Cerebras [API](#) with model identifier `zai-glm-4.7` (GLM 4.7), configured through `CEREBRAS_MODEL` in the environment. The client module `cerebras_client.py` submits chat completions with tool definitions generated from the registry and prompt autogen files under `prompts/tool_calling/`. GLM 4.7 supports *native parallel* `tool_calls` in a single completion when the model can plan an entire implement batch at once; the provider module `llm_provider.py` records this capability and applies Cerebras free-tier rate limits (requests and tokens per minute).

Temperature and *top-p* are set conservatively (`LLM_TEMPERATURE` default 0.5) to favour deterministic tool selection over creative variation. When Agentic mode is active and the server allows it, tool schemas are attached to the completion request; the host parses returned `tool_calls`, validates arguments, applies phase guards, and either executes local tools immediately or enqueues browser tools for the extension. Streaming responses (`streaming.py`) support incremental user-interface updates in Ask and Code modes; Agentic batches may still stream textual narration while tools are queued.